

Break Statement in C# with Examples

In this article, I am going to discuss **Break Statement in C#** Language with Examples. Please read our previous articles, where we discussed [For loop in C#](#) with Examples. Before understanding the Break Statement, in this article, first I will discuss what are Jump Statements and when and how to use Jump Statements in C# Language and then I will discuss Break statements with Examples.

What are Jump Statements in C# Language?

The Jump Statements in C# are used to transfer control from one point or location or statement to another point or location or statement in the program due to some specified condition while executing the program.

The Jump Statements in C# Language are used to modify the behavior of conditional (if, else, switch) and iterative (for, while, and do-while) statements. The Jump Statements allow us to exit a loop, and start the next iteration, or explicitly transfer the program control to a specified location in your program. C# supports the following jump statements:

1. **break**
2. **continue**
3. **goto**
4. **return** (In the Function section we will discuss the return statement)
5. **throw** (In the Exception Handling section we will discuss the throw statement)

Break Statement in C# Language:

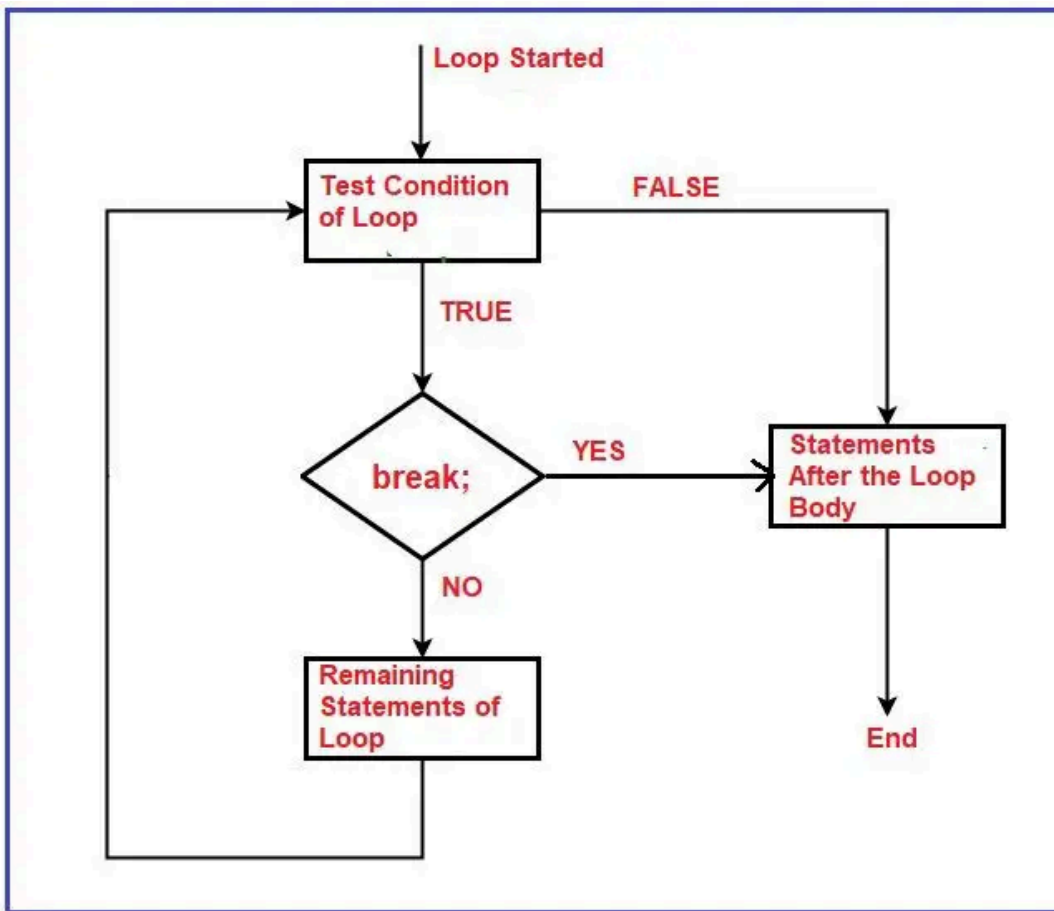
In C#, the break is a keyword. By using the break statement, we can terminate either the loop body or the switch body. The most important point that you need to keep in mind is the use of a break statement is optional but if you want to use then the break statement should be placed either within the loop body or switch body.

Now, the question is when should we use the break statement? The answer is when we know the maximum number of repetitions of a loop but if some condition is there where we need to terminate the loop body then we need to use the break statement. We have already discussed the use of the break statement in the switch case. In C#, every case statement should be ended with a break statement, else we will get compile time error. When the break statement with a case block is executed, it will terminate the switch block.

That means the Break Statement in C# provides a convenient way to immediately exit from a loop (For, While, Do While, Foreach), or Switch Case statement. The break statement ends the loop immediately when it is encountered. The break statement is almost always used with the if...else statement inside the loop body. If this is not clear at the moment, don't worry we will explain these things in multiple examples.

Syntax: **break;**

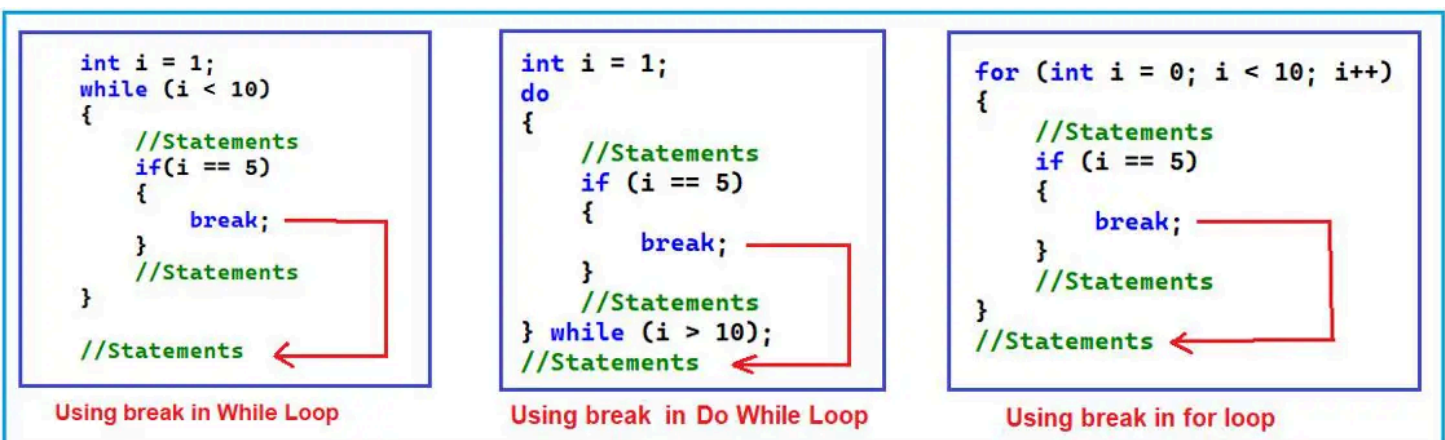
Flowchart of Break Statement:



When it encounters the break statement inside a loop body or switch body, then immediately it terminates the loop and switch execution and executes the statements which are present after the loop body or switch body. But if the break statement is not executed, then the statements which are present after the break statement will be executed and then it will continue its execution with the next iteration of the loop. If this is not clear at the moment then don't worry, we will understand this with multiple examples.

How does the break statement work in C# language?

In our switch case conditional statement, we have discussed how to use the break statement. Now, let us understand how to use the break statement inside the loop and how the break statement exactly works in C#. To understand this, please have a look at the following image. Here, I am showing how to use the break statement inside do while, while, and for loop and how exactly the break statement work.



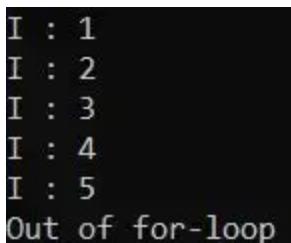
If you notice the above code, we have written the if conditional statement inside the loop body, and within the if condition block, we have written the break statement. So, when the loop executes, in each iteration, the if condition will be checked and if the condition is false, then it will execute the statements which are present after the if block and continue with the next iteration. Now, what happens when the if condition is true? Once the if condition is evaluated to true, then the if block will be executed, and once the break statement within the if block is executed, it immediately terminates the loop, and the statements which are present after the loop block will be executed.

Example to Understand Break Statement in C# Language:

In the below example, we have provided the condition for the loop to be executed 10 times i.e. starting from I value 1 to 10. But our requirement is when the I value becomes 5, we need to terminate the loop. In this case, we need to write if condition inside the loop body and check whether the current I value is equal to 5 or not. If it is not equal to 5, then continue the execution of the for loop and execute the next iteration. But if the I value is 5, then the, if condition will return true, and, in that case, the break statement will be executed and once the break statement is executed, it will immediately terminate the loop body. So, the next statement which is present after the loop body is going to execute.

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine($"I : {i}");
                if (i == 5)
                {
                    break;
                }
            }
            Console.WriteLine("Out of for-loop");
            Console.ReadKey();
        }
    }
}
```

Output:



```
I : 1
I : 2
I : 3
I : 4
I : 5
Out of for-loop
```

C# Break Statement with Inner Loop:

The break statement in C# terminates the closest enclosing iteration statement (for, for each, while, or do loop) or switch statement. If we place the break statement inside the nested loop i.e. inside the inner loop, then the break statement will terminate only the innermost loop that contains it.

For a better understanding, please have a look at the below example. Here, the outer loop will execute 5 times. Again, for each outer loop iteration, we have written the inner for loop condition to execute 5 times. But, inside the inner loop body, we have written the break statement using the if condition, and when the inner value is equal to 3, it will break the inner loop.

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int outer = 1; outer <= 5; outer++)
            {
                Console.WriteLine($"Outer: {outer}");
                for (int inner = 1; inner <= 5; inner++)
                {
                    if (inner > 3)
                    {
                        break;
                    }
                    Console.Write($"  Inner: {inner}");
                }
                Console.WriteLine();
            }
            Console.ReadKey();
        }
    }
}
```

The point that you need to remember is the break statement will terminate the inner loop body only. Once the inner loop is terminated, it will continue the execution of the outer loop body and will also continue with the execution of the next iteration of the outer loop. When you execute the above code, you will get the following output.

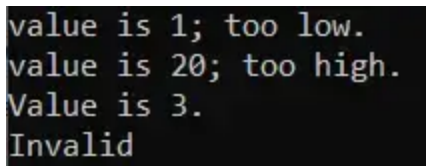
```
Outer: 1
  Inner: 1  Inner: 2  Inner: 3
Outer: 2
  Inner: 1  Inner: 2  Inner: 3
Outer: 3
  Inner: 1  Inner: 2  Inner: 3
Outer: 4
  Inner: 1  Inner: 2  Inner: 3
Outer: 5
  Inner: 1  Inner: 2  Inner: 3
```

Break Statement with Loop and Switch Statements in C#:

When we use the switch statement inside a loop, the break statement at the end of a switch section transfers controls only out of the switch statement, not out of the loop statement. The loop that contains the switch statement is unaffected. For a better understanding, please have a look at the following example.

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            double[] measurements = { 1, 20, 3, double.NaN };
            foreach (double measurement in measurements)
            {
                switch (measurement)
                {
                    case 1:
                        Console.WriteLine($"value is {measurement}; too low.");
                        break;
                    case 20:
                        Console.WriteLine($"value is {measurement}; too high.");
                        break;
                    case double.NaN:
                        Console.WriteLine("Invalid");
                        break;
                    default:
                        Console.WriteLine($"Value is {measurement}.");
                        break;
                }
            }
            Console.ReadKey();
        }
    }
}
```

Output:

A screenshot of a terminal window showing the output of the C# program. The output consists of four lines: "value is 1; too low.", "value is 20; too high.", "Value is 3.", and "Invalid". The text is white on a black background.

```
value is 1; too low.
value is 20; too high.
Value is 3.
Invalid
```

Note: In General, we should minimize the use of break statements in loops. The switch statement is an exception in this regard where it is necessary to use the “break statement” after every case. Otherwise, there may be a logical error. while writing loops, we should try to execute the loops with the condition test and should try to avoid the “break statement”.

Some tricky questions related to C# Break Statement:

Question1: What will be the output in the below program?

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 1;
            while (a <= 10)
            {
                Console.Write($"{a} ");
                if (a > 3)
                    break;
                a++;
            }
            Console.ReadKey();
        }
    }
}
```

Output: 1 2 3 4

This is because whenever the value of a become 4 then the condition becomes true then the break statement will be executed. Whenever the break statement is executed automatically control will pass outside of the loop body.

Question2: What will be the output in the below program?

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 2;
            while (a <= 20)
            {
                Console.Write($"{a} ");
                a += 2;
                if (a >= 8)
                    break;
            }
            Console.ReadKey();
        }
    }
}
```

```
}  
}
```

Output: 2 4 6

Question3: What will be the output in the below program?

```
using System;  
namespace JumpStatementDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int a = 15;  
            while (a >= 3)  
            {  
                a -= 2;  
                Console.Write($"{a} ");  
                if (a <= 9)  
                    break;  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

Output: 13 11 9

Question4: What will be the output in the below program?

```
using System;  
namespace JumpStatementDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int a = 15;  
            while (a >= 5)  
            {  
                Console.Write($"{a} ");  
                if (a <= 10);  
                break;  
                a -= 2;  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

```
}
```

Output: 15

Note: When the semicolon (;) is available at the end of the line then it became a dummy condition that the break statement is placed directly outside the condition in the loop.

Question5: What will be the output in the below program?

```
using System;
namespace JumpStatementDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 8;
            while (a <= 80) ;
            {
                Console.Write($"{a} ");
                if (a >= 20) ;
                break;
                a += 2;
            }
            Console.ReadKey();
        }
    }
}
```

Output: Error CS0139 No enclosing loop out of which to break or continue

Note: When the semicolon is available at the end of the while then it becomes a dummy loop. When the dummy loop is created then the compiler will create a new body without any statements and the current body becomes outside so automatically break become outside and as we know we cannot use break outside of the loop body.

In the next article, I am going to discuss [Continue Statement in C#](#) with Examples. Here, in this article, I try to explain the **Break Statement in C# Language** with Examples. I hope you enjoy this Break Statement in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.